

Mysql 安装部署文档



北京首钢自动化信息技术有限公司

2024 年 9 月



版 本 号	创 建 时 间	作 者	受控状态	审核一	审核二
1.0	2024-09-01	刘壮			

文件修改记录

序号	修改后版本号	修改日期	修改人	修改原因/说明	修改内容

目 录

1	Mysql 安装.....	4
1.1	操作环境.....	4
1.2	主机信息检查.....	5
1.2.1	网络相关检查.....	5
1.2.2	磁盘相关检查.....	6
1.2.3	CPU、内存检查.....	7
1.2.4	目录相关检查.....	7
1.3	MySQL 数据库安装.....	7
1.4	主从环境的搭建.....	19
1.5	安装 MHA NODE.....	20
1.6	安装 MHA MANAGER.....	21
1.7	配置节点互信.....	22
1.8	配置 MHA.....	23
1.9	检查状态.....	24
1.10	启停 MHA MANAGER 监控.....	28
1.11	主从恢复脚本.....	29
1.12	创建自动清除 RELAY LOG 脚本.....	29
1.13	使用 KEEPALIVED 引入 VIP.....	30

1 Mysql 安装

序号	主机名	地址	安装目录	部署基础软件	备注
1	Mysql-01	192.168.4.47/24	/u01	mysql	
2	Mysql-02	192.168.4.48/24	/u01	mysql	
3	Mysql-03	192.168.4.49/24	/u01	mysql	
4		192.168.4.50/24			浮动 IP
5	Mysql-04	192.168.4.51/24	/u01	mysql	
6	Mysql-05	192.168.4.52/24	/u01	mysql	
7	Mysql-06	192.168.4.53/24	/u01	mysql	
8		192.168.4.54/24			浮动 IP

1.1操作环境

CentOS Linux release 7.7.1908 (Core)

1.2 主机信息检查

1.2.1 网络相关检查

- 1) 检查确认主机名，与规划文档进行核对确认主机名是否需要修改。

检查:

```
[root@localhost ~]# hostnamectl
Static hostname: localhost.localdomain
Icon name: computer-vm
Chassis: vm
Machine ID: 8633a1bd91e447908a40c0df8eaec169
Boot ID: f293dbd6c7d642439394156c7cffe519
Virtualization: vmware
Operating System: CentOS Linux 7 (Core)
CPE OS Name: cpe:/o:centos:centos:7
Kernel: Linux 3.10.0-514.el7.x86_64
Architecture: x86-64
```

修改:

```
[root@localhost ~]# hostnamectl --static set-hostname mysql-01
[root@sitpp1 ~]# hostnamectl
Static hostname: mysql-01
Icon name: computer-vm
Chassis: vm
Machine ID: 0fb41200949e3c54b8d1dea4e83c3074
Boot ID: b9d1ecb1799b4f34a32df70e168cbda0
Virtualization: kvm
Operating System: CentOS Linux 7 (Core)
CPE OS Name: cpe:/o:centos:centos:7
Kernel: Linux 3.10.0-1062.el7.x86_64
Architecture: x86-64
```

- 2) 执行以下命令确认是否删除虚拟网络。

检查方法如下:

```
[root@localhost ~]# ip addr list
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 52:54:00:12:34:56 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.44/24 brd 192.168.2.255 scope global noprefixroute dynamic eth0
        valid_lft 76872sec preferred_lft 76872sec
    inet6 fe80::5054:ccff:fea3:dc56/64 scope link
        valid_lft forever preferred_lft forever
```

3) 时钟同步

查看时区

```
# timedatectl

Local time: Thu 2020-05-21 09:40:07 CST
Universal time: Thu 2020-05-21 01:40:07 UTC
RTC time: Thu 2020-05-21 01:40:06
Time zone: Asia/Shanghai (CST, +0800)
NTP enabled: n/a
NTP synchronized: no
RTC in local TZ: no
DST active: n/a
```

设置时区

```
# timedatectl set-timezone Asia/Shanghai
```

配置时间同步

chrony 方式配置

```
vim /etc/chrony.conf
#添加以下内容
server 192.168.147.80 iburst #北京
server ntp1.aliyun.com iburst #阿里云
[root@localhost]# systemctl start chronyd.service
[root@localhost]# systemctl restart chronyd.service
[root@localhost]# systemctl status chronyd.service
[root@localhost]# systemctl enable chronyd.service
[root@localhost]# chronyc sources -v
[root@localhost]# chronyc sourcestats -v
```

4) SELINUX

```
[root@localhost /]# sed -i 's/SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config
[root@localhost /]# grep SELINUX=disabled /etc/selinux/config
SELINUX=disabled
[root@localhost /]# setenforce 0
```

5) 防火墙设置

firewall 关闭

```
[root@localhost /]# systemctl stop firewalld.service
[root@localhost /]# systemctl disable firewalld.service
```

1.2.2 磁盘相关检查

确认磁盘大小与规划大小是否一致，确认分区是否一致

```
[root@localhost ~]# fdisk -l
```

1.2.3 CPU、内存检查

确认 CPU、内存与规划大小是否一致，确认分区是否一致

查看 CPU 个数

```
# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
[root@CICD-01 ~]# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
8
```

查看 CPU 核数

```
# cat /proc/cpuinfo | grep "cpu cores" | uniq
[root@CICD-01 ~]# cat /proc/cpuinfo | grep "cpu cores" | uniq
cpu cores      : 1
```

查看 CPU 型号

```
# cat /proc/cpuinfo | grep 'model name' | uniq
[root@CICD-01 ~]# cat /proc/cpuinfo | grep 'model name' | uniq
model name     : QEMU Virtual CPU
```

查看内存总数

```
#cat /proc/meminfo | grep MemTotal
[root@CICD-01 ~]# cat /proc/meminfo | grep MemTotal
MemTotal:      16265588 kB    //内存 16G
```

1.2.4 目录相关检查

检查介质目录

```
[root@localhost ~]# ll /software
```

创建介质目录将相关介质放置该目录备用

```
[root@localhost ~]# mkdir /software
```

1.3 Mysql 数据库安装

上传 mysql-5.7.21-linux-glibc2.12-x86_64.tar.gz 到/opt 目录，执行下面命令：

```
tar -zxvf mysql-5.7.21-linux-glibc2.12-x86_64.tar.gz
mkdir -p /u01/data
mv mysql-5.7.21-linux-glibc2.12-x86_64 /u01/mysql
```

```
groupadd mysql
useradd -r -s /sbin/nologin -g mysql mysql -d /usr/local/mysql
usermod mysql -d /u01/mysql
cd /u01/
chown -R mysql:mysql data/ mysql/
cd /u01/mysql/
bin/mysqld --initialize --user=mysql --basedir=/u01/mysql --datadir=/u01/data
bin/mysql_ssl_rsa_setup --datadir=/u01/data
cd /u01/mysql/support-files/
cp mysql.server /etc/init.d/mysql
mkdir -p /u01/mysql/logs
touch /u01/mysql/logs/mysql.log
chown -R mysql:mysql /u01/data/ /u01/mysql/
/etc/init.d/mysqld start
mysql> set password=password("123456");
mysql> grant all privileges on *.* to root@'%' identified by '123456';
```

配置文件内容如下:

/etc/my.cnf 文件

```
[root@mysql-01 ~]# cat /etc/my.cnf

[mysqld]
basedir = /u01/mysql
datadir = /u01/data
port = 6133
socket = /tmp/mysql.sock
character-set-server = utf8
# Disabling symbolic-links is recommended to prevent assorted security risks
symbolic-links=0
# Settings user and group are ignored when systemd is used.
# If you need to run mysqld under a different user or group,
# customize your systemd unit file for mariadb according to the
lower_case_table_names = 1
# instructions in http://fedoraproject.org/wiki/Systemd
innodb_buffer_pool_size=8G
innodb_thread_concurrency=16
innodb_log_buffer_size=20M
innodb_log_file_size=500M
max_allowed_packet=256M
sql_mode=NO_ENGINE_SUBSTITUTION,STRICT_TRANS_TABLES
sort_buffer_size = 16777216
key_cache_block_size = 2048
query_cache_size = 4194304
read_buffer_size = 262144
read_rnd_buffer_size = 524288
```



```
slave_pending_jobs_size_max = 33554432
sort_buffer_size = 524288
thread_cache_size = 18
max_allowed_packet = 41943040
wait_timeout=31536000
interactive_timeout=31536000
wait_timeout = 300

skip-name-resolve
#skip-grant-tables
max_connections=1000
max_connect_errors=200
sync_binlog=4
server-id=47
log-bin=mysql-bin
expire_logs_days=7
slow-query-log = ON      #慢查询开关
long_query_time = 3
default-time-zone = '+08:00'
log_timestamps=SYSTEM
log_bin_trust_function_creators=1

# slave
slave-parallel-type=LOGICAL_CLOCK
slave-parallel-workers=8
master_info_repository=TABLE
relay_log_info_repository=TABLE
relay_log_recovery=1

[mysqld_safe]
log-error=/u01/mysql/logs/mysql.log
pid-file=/u01/mysql/tmp/mysql.pid

#
# include all files from the config directory
#
!includedir /etc/my.cnf.d
sort_buffer_size = 524288
thread_cache_size = 18
max_connections = 500
max_allowed_packet = 41943040
wait_timeout=31536000
interactive_timeout=31536000
```

```
wait_timeout = 300

skip-name-resolve
#skip-grant-tables
max_connections=1000
max_connect_errors=200
log-bin=mysql-bin
server-id=56
expire_logs_days=7
slow-query-log = ON    #慢查询开关
long_query_time = 10

[mysqld_safe]
log-error=/u01/mysql/log/mysql.log
pid-file=/u01/mysql/tmp/mysql.pid

#
# include all files from the config directory
#
!includedir /etc/my.cnf.d
```

/etc/init.d/mysqld 文件

```
[root@mysql-01 ~]# cat /etc/init.d/mysqld
#!/bin/sh

# Copyright Abandoned 1996 TCX DataKonsult AB & Monty Program KB & Detron HB
# This file is public domain and comes with NO WARRANTY of any kind

# MySQL daemon start/stop script.

# Usually this is put in /etc/init.d (at least on machines SYSV R4 based
# systems) and linked to /etc/rc3.d/S99mysql and /etc/rc0.d/K01mysql.
# When this is done the mysql server will be started when the machine is
# started and shut down when the systems goes down.

# Comments to support chkconfig on RedHat Linux
# chkconfig: 2345 64 36
# description: A very fast and reliable SQL database engine.

# Comments to support LSB init script conventions
### BEGIN INIT INFO
# Provides: mysql
# Required-Start: $local_fs $network $remote_fs
# Should-Start: ypbind nsd ldap ntpd xntpd
```

```
# Required-Stop: $local_fs $network $remote_fs
# Default-Start: 2 3 4 5
# Default-Stop: 0 1 6
# Short-Description: start and stop MySQL
# Description: MySQL is a very fast and reliable SQL database engine.
### END INIT INFO

# If you install MySQL on some other places than /usr/local/mysql, then you
# have to do one of the following things for this script to work:
#
# - Run this script from within the MySQL installation directory
# - Create a /etc/my.cnf file with the following information:
#   [mysqld]
#   basedir=<path-to-mysql-installation-directory>
# - Add the above to any other configuration file (for example ~/.my.ini)
#   and copy my_print_defaults to /usr/bin
# - Add the path to the mysql-installation-directory to the basedir variable
#   below.
#
# If you want to affect other MySQL variables, you should make your changes
# in the /etc/my.cnf, ~/.my.cnf or other MySQL configuration files.

# If you change base dir, you must also change datadir. These may get
# overwritten by settings in the MySQL configuration files.

basedir=/u01/mysql
datadir=/u01/data

# Default value, in seconds, afterwhich the script should timeout waiting
# for server start.
# Value here is overridden by value in my.cnf.
# 0 means don't wait at all
# Negative numbers mean to wait indefinitely
service_startup_timeout=900

# Lock directory for RedHat / SuSE.
lockdir='/var/lock/subsys'
lock_file_path="$lockdir/mysql"

# The following variables are only set for letting mysql.server find things.

# Set some defaults
mysqld_pid_file_path=
if test -z "$basedir"
```

```
then
    basedir=/u01/mysql
    bindir=/u01/mysql/bin
    if test -z "$datadir"
    then
        datadir=/u01/data
    fi
    sbindir=/u01/mysql/bin
    libexecdir=/u01/mysql/bin
else
    bindir="$basedir/bin"
    if test -z "$datadir"
    then
        datadir="$basedir/data"
    fi
    sbindir="$basedir/sbin"
    libexecdir="$basedir/libexec"
fi

# datadir_set is used to determine if datadir was set (and so should be
# *not* set inside of the --basedir= handler.)
datadir_set=

#
# Use LSB init script functions for printing messages, if possible
#
lsb_functions="/lib/lsb/init-functions"
if test -f $lsb_functions ; then
    . $lsb_functions
else
    log_success_msg()
    {
        echo " SUCCESS! @$"
    }
    log_failure_msg()
    {
        echo " ERROR! @$"
    }
fi

PATH="/sbin:/usr/sbin:/bin:/usr/bin:$basedir/bin"
export PATH

mode=$1    # start or stop
```

```
[ $# -ge 1 ] && shift

other_args="$*" # uncommon, but needed when called from an RPM upgrade action
# Expected: "--skip-networking --skip-grant-tables"
# They are not checked here, intentionally, as it is the responsibility
# of the "spec" file author to give correct arguments only.

case `echo "testing\c"` in
  *c*,-n*) echo_n= echo_c= ;;
  *c*,*) echo_n=-n echo_c= ;;
  *) echo_n= echo_c='\c' ;;
esac

parse_server_arguments() {
  for arg do
    case "$arg" in
      --basedir=*) basedir=`echo "$arg" | sed -e 's/^\(=\)*=/'`
                    bindir="$basedir/bin"
                    if test -z "$datadir_set"; then
                      datadir="$basedir/data"
                    fi
                    sbindir="$basedir/sbin"
                    libexecdir="$basedir/libexec"
                    ;;
      --datadir=*) datadir=`echo "$arg" | sed -e 's/^\(=\)*=/'`
                    datadir_set=1
                    ;;
      --pid-file=*) mysqld_pid_file_path=`echo "$arg" | sed -e 's/^\(=\)*=/'` ;;
      --service-startup-timeout=*) service_startup_timeout=`echo "$arg" | sed -e
's/^\(=\)*=/'` ;;
    esac
  done
}

wait_for_pid () {
  verb="$1" # created | removed
  pid="$2" # process ID of the program operating on the pid-file
  pid_file_path="$3" # path to the PID file.

  i=0
  avoid_race_condition="by checking again"
```

```
while test $i -ne $service_startup_timeout ; do

    case "$verb" in
        'created')
            # wait for a PID-file to pop into existence.
            test -s "$pid_file_path" && i='' && break
            ;;
        'removed')
            # wait for this PID-file to disappear
            test ! -s "$pid_file_path" && i='' && break
            ;;
        *)
            echo "wait_for_pid () usage: wait_for_pid created|removed pid pid_file_path"
            exit 1
            ;;
    esac

    # if server isn't running, then pid-file will never be updated
    if test -n "$pid"; then
        if kill -0 "$pid" 2>/dev/null; then
            : # the server still runs
        else
            # The server may have exited between the last pid-file check and now.
            if test -n "$avoid_race_condition"; then
                avoid_race_condition=""
                continue # Check again.
            fi

            # there's nothing that will affect the file.
            log_failure_msg "The server quit without updating PID file ($pid_file_path)."
            return 1 # not waiting any more.
        fi
    fi

    echo $echo_n ".$echo_c"
    i=`expr $i + 1`
    sleep 1

done

if test -z "$i" ; then
    log_success_msg
    return 0
else
```

```
log_failure_msg
return 1
fi
}

# Get arguments from the my.cnf file,
# the only group, which is read from now on is [mysqld]
if test -x "$bindir/my_print_defaults"; then
    print_defaults="$bindir/my_print_defaults"
else
    # Try to find basedir in /etc/my.cnf
    conf=/etc/my.cnf
    print_defaults=
    if test -r $conf
    then
        subpat='^[^=]*basedir[^=]*=\\(\\.\\.\\)'
        dirs=`sed -e "/$subpat/!d" -e 's/\\/\\1/' $conf`
        for d in $dirs
        do
            d=`echo $d | sed -e 's/[ ]//g'`
            if test -x "$d/bin/my_print_defaults"
            then
                print_defaults="$d/bin/my_print_defaults"
                break
            fi
        done
    fi

    # Hope it's in the PATH ... but I doubt it
    test -z "$print_defaults" && print_defaults="my_print_defaults"
fi

#
# Read defaults file from 'basedir'. If there is no defaults file there
# check if it's in the old (depricated) place (datadir) and read it from there
#

extra_args=""
if test -r "$basedir/my.cnf"
then
    extra_args="-e $basedir/my.cnf"
fi

parse_server_arguments ` $print_defaults $extra_args mysqld server mysql_server`
```

```
mysql.server`

#
# Set pid file if not given
#
if test -z "$mysqld_pid_file_path"
then
    mysqld_pid_file_path=$datadir/`hostname`.pid
else
    case "$mysqld_pid_file_path" in
        /* ) ;;
        * ) mysqld_pid_file_path="$datadir/$mysqld_pid_file_path" ;;
    esac
fi

case "$mode" in
    'start')
        # Start daemon

        # Safeguard (relative paths, core dumps..)
        cd $basedir

        echo $echo_n "Starting MySQL"
        if test -x $bindir/mysqld_safe
        then
            # Give extra arguments to mysqld with the my.cnf file. This script
            # may be overwritten at next upgrade.
            $bindir/mysqld_safe --datadir="$datadir" --pid-file="$mysqld_pid_file_path"
$other_args >/dev/null &
            wait_for_pid created "$!" "$mysqld_pid_file_path"; return_value=$?

            # Make lock for RedHat / SuSE
            if test -w "$lockdir"
            then
                touch "$lock_file_path"
            fi

            exit $return_value
        else
            log_failure_msg "Couldn't find MySQL server ($bindir/mysqld_safe)"
        fi
        ;;

    'stop')
```



```
# Stop daemon. We use a signal here to avoid having to know the
# root password.

if test -s "$mysqld_pid_file_path"
then
    # signal mysqld_safe that it needs to stop
    touch "$mysqld_pid_file_path.shutdown"

    mysqld_pid=`cat "$mysqld_pid_file_path"`

    if (kill -0 $mysqld_pid 2>/dev/null)
    then
        echo $echo_n "Shutting down MySQL"
        kill $mysqld_pid
        # mysqld should remove the pid file when it exits, so wait for it.
        wait_for_pid removed "$mysqld_pid" "$mysqld_pid_file_path"; return_value=$?
    else
        log_failure_msg "MySQL server process #$mysqld_pid is not running!"
        rm "$mysqld_pid_file_path"
    fi

    # Delete lock for RedHat / SuSE
    if test -f "$lock_file_path"
    then
        rm -f "$lock_file_path"
    fi
    exit $return_value
else
    log_failure_msg "MySQL server PID file could not be found!"
fi
;;

'restart')
    # Stop the service and regardless of whether it was
    # running or not, start it again.
    if $0 stop $other_args; then
        $0 start $other_args
    else
        log_failure_msg "Failed to stop running server, so refusing to try to start."
        exit 1
    fi
    ;;

'reload'|'force-reload')
```

```
if test -s "$mysqld_pid_file_path" ; then
    read mysqld_pid < "$mysqld_pid_file_path"
    kill -HUP $mysqld_pid && log_success_msg "Reloading service MySQL"
    touch "$mysqld_pid_file_path"
else
    log_failure_msg "MySQL PID file could not be found!"
    exit 1
fi
;;
'status')
# First, check to see if pid file exists
if test -s "$mysqld_pid_file_path" ; then
    read mysqld_pid < "$mysqld_pid_file_path"
    if kill -0 $mysqld_pid 2>/dev/null ; then
        log_success_msg "MySQL running ($mysqld_pid)"
        exit 0
    else
        log_failure_msg "MySQL is not running, but PID file exists"
        exit 1
    fi
else
    # Try to find appropriate mysqld process
    mysqld_pid=`pidof $libexecdir/mysqld`

    # test if multiple pids exist
    pid_count=`echo $mysqld_pid | wc -w`
    if test $pid_count -gt 1 ; then
        log_failure_msg "Multiple MySQL running but PID file could not be found"
        ($mysqld_pid)"
        exit 5
    elif test -z $mysqld_pid ; then
        if test -f "$lock_file_path" ; then
            log_failure_msg "MySQL is not running, but lock file ($lock_file_path) exists"
            exit 2
        fi
        log_failure_msg "MySQL is not running"
        exit 3
    else
        log_failure_msg "MySQL is running but PID file could not be found"
        exit 4
    fi
fi
;;
*)
```

```
# usage
basename=`basename "$0"`
echo "Usage: $basename {start|stop|restart|reload|force-reload|status} [ MySQL
server options ]"
exit 1
;;
esac

exit 0
```

1.4主从环境的搭建

1.修改从服务器的配置文件/etc/my.cnf，在 mysqld 里添加一下属性

```
[mysqld]
log-bin=mysql-bin    //[必须]启用二进制日志
server-id=129        //[必须]服务器唯一 ID，默认是 1，一般取 IP 最后一段，如 IP: 192.168.80.129
```

2.从服务器也做以上操作，添加完之后，重新启动两个数据库。

3.主服务器上进行操作

(1) 创建用户并授权

```
mysql -u root -p
mysql>grant replication slave on *.* to root@'%' identified by '123456';
mysql>flush privileges;
```

(2) 查询主数据库的状态

```
mysql> show master status;
```

记下 File 以及 Position 的值，在后面进行从服务器操作的时候需要使用。

```
mysql> show master status;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql-bin.000002 | 6770614 |              |                  |                  |
+-----+-----+-----+-----+-----+
1 row in set (0.03 sec)
```

4.在从服务器上操作

(1) mysql 登录并执行同步的 sql 语句，最后并启动 slave

```
#mysql -uroot -p
mysql>change master to master_host='192.168.83.130',master_port =
6133,master_user='root',master_password='123456',master_log_file='mysql-bin.000002',ma
ster_log_pos=741;
```

启动了 slave 同步功能

```
mysql> start slave;
```

(2) 主从同步检查

执行如下语句

```
mysql> show slave status\G
```

来检查从服务器的同步情况，一个正常的输出结果应该如下面的形式：

Slave_IO 进程以及 slave_SQL 进程都必须正常运行，在状态输出中表现为：“slave”；否则都是不正确的状态（如一个值是 Yes，另一个是 No 则不行）。

如果主数据库服务器已经存在用户数据，那么在进行主从复制时，需要做以下处理：

(1) 主数据库锁表操作，不让数据再进行写入动作。

```
mysql> flush tables with read lock;
```

(2) 查看主数据库的状态，记下 File 以及 Position 的值，以备从服务器使用。

```
mysql> show master status;
```

(3) 把主服务器的数据文件复制到从服务器，最好先用 tar 归档压缩处理一下

(4) 取消主数据库锁定

```
mysql> unlock tables;
```

需要注意的几点：

- 不要将 read_only=1 写进从库的配置文件，因为主库宕机时，从库要提升为主库接受写请求 `mysql -e"set global read_only=1"`
- 主从节点复制的过滤规则要相同，即 `binlog_do_db` 与 `binlog_ignore_db` 参数主从配置需要相同
- 从节点需要修改配置参数 `relay_log_purge=0`，即关闭中继日志的清除

#采用命令方式将从库设为只读，不要将该参数写进配置文件中

```
mysql -e"set global read_only=1"
```

#关闭中继日志的清除

```
mysql -e"set global relay_log_purge=0"
```

1.5 安装 MHA node

所有节点（数据库 master，slave，MHA manager 节点）都需要安装 MHA node。因为 MHA manager 也需要依赖 MHA node。

#下载 MHA-node <https://github.com/yoshinorim/mha4mysql-node/releases>

#安装依赖，这里遇到的报错大多事因为没有安装依赖环境

```
yum install -y perl-DBD-MySQL perl-ExtUtils-MakeMaker perl-CPAN
```

```
#解压
tar -zxvf mha4mysql-node-0.58.tar.gz
#移动到/u01/mha4mysql-node-0.58 目录下
mv mha4mysql-node-0.58 /u01/mha4mysql-node-0.58
#编译
cd /usr/local/mha4mysql-node-0.58
perl Makefile.PL
#make 并安装
make
make install
##或者用 rpm 包安装
rpm -ivh mha4mysql-node-0.58-0.el7.centos.noarch.rpm
```

安装完成后 会在/usr/local/bin/生成以下脚本文件，Node 脚本说明：(这些工具通常由 MHA Manager 的脚本触发，无需人为操作)

```
save_binary_logs          //保存和复制 master 的二进制日志
apply_diff_relay_logs     //识别差异的中继日志事件并将其差异的事件应用于其他 slave
filter_mysqlbinlog        //去除不必要的 ROLLBACK 事件 (MHA 已不再使用这个工具)
purge_relay_logs          //清除中继日志 (不会阻塞 SQL 线程)
```

1.6 安装 MHA manager

只需要在 10.68.28.57 节点安装 MHA-manager

```
#下载 MHA-manager
https://github.com/yoshinorim/mha4mysql-manager/releases
#配置 yum 源
wget -O /etc/yum.repos.d/CentOS-Base.repo http://mirrors.aliyun.com/repo/Centos-7.repo
sed -i 's/$releasever/7/g' /etc/yum.repos.d/CentOS-Base.repo
yum repolist
yum -y install yum-utils
wget -O /etc/yum.repos.d/epel.repo http://mirrors.aliyun.com/repo/epel-7.repo
yum-config-manager --add-repo
http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
yum makecache
#安装依赖包
yum install perl-DBD-MySQL perl-Config-Tiny perl-Log-Dispatch perl-Parallel-ForkManager
-y
#解压
tar -zxf mha4mysql-manager-0.58.tar.gz
#移动目录
mv mha4mysql-manager-0.58 /u01/mha4mysql-manager-0.58
# 编译安装
```

```
cd /usr/local/mha4mysql-manager-0.58/  
perl Makefile.PL  
make  
make install  
  
##或者用 rpm 包安装  
rpm -ivh mha4mysql-manager-0.58-0.e17.centos.noarch.rpm
```

安装完成后，在/usr/local/bin 会多出以下相关的命令脚本：

```
masterha_check_rep1 //检查 MySQL 复制健康状况  
masterha_check_ssh //检查 ssh 健康状况  
masterha_check_status //检测当前 MHA 运行状态  
masterha_conf_host //添加或者删除配置的 server 信息  
masterha_manager //启动 MHA  
masterha_master_monitor //检测 master 是否宕机  
masterha_master_switch //控制故障转移（自动或者手动）  
masterha_secondary_check //如果从 manager 节点发现  
masterha_stop //停止 MHA
```

在目录下会有相关脚本

```
master_ip_failover //自动切换时 vip 管理的脚本，不是必须，如果我们使用 keepalived 的，我们可以自己  
编写脚本完成对 vip 的管理，比如监控 mysql，如果 mysql 异常，我们停止 keepalived 就行，这样 vip 就会自动  
漂移  
master_ip_online_change //在线切换时 vip 的管理，不是必须，同样可以自行编写简单的 shell 完成  
power_manager //故障发生后关闭主机的脚本，不是必须  
send_report //因故障切换后发送报警的脚本，不是必须，可自行编写简单的 shell 完成
```

安装部分参考 Git 文档 <https://github.com/yoshinorim/mha4mysql-manager/wiki/Installation>

1.7 配置节点互信

后面为方便阐述，用 47 代表 192.168.4.47 服务器。

```
#所有节点都执行互信  
##生成私钥和秘钥  
ssh-keygen -t rsa (一路回车)  
#与另外两个节点进行互信  
ssh-copy-id 192.168.83.131  
ssh-copy-id 192.168.83.132
```

1.8 配置 MHA

为保证 MHA 正常工作，需要配置 MHA 的配置文件，为参数设置合理正确的值，这些参数包括服务器 IP，数据库用户名密码，工作目录与日志等。MHA manager 有两个配置文件模板，在路径 \$MHA_BASE/samples/conf/ 下的 app50.cnf 和 masterha_default.cnf。

##三个节点创建目录

```
mkdir /etc/mha
```

```
mkdir /u01/mha
```

```
mkdir /u01/mha_tmp
```

##MHA 管理节点编写切换脚本（本次用的是三节点）

app50.cnf #是对某个复制组的配置文件。

masterha_default.cnf #MHA manager 的全局配置文件，可以通过这一个配置文件管理多个复制组。

app50.cnf 配置文件，这里先注释掉故障切换脚本，修改好脚本后再打开注释。

创建数据库监控用户，本次实验用的是 root 用户

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@ '%' identified by '123456';
flush privileges;
```

```
# vim /etc/mha/app50.cnf
```

```
[server default]
```

```
#mha manager 日志文件
```

```
manager_log=/u01/mha/app50/manager.log
```

```
#manager 工作目录
```

```
manager_workdir=/u01/mha/app50
```

```
#master 节点存放 binlog 日志路径,以便 MHA 找到 binlog, 这里就是 MySQL 的数据目录
```

```
master_binlog_dir=/u01/data
```

```
#发生切换时 slave 节点 binlog 日志存放路径
```

```
remote_workdir=/u01/mha_tmp
```

```
#自动切换脚本
```

```
#master_ip_failover_script=/usr/local/bin/master_ip_failover
```

```
#手动切换脚本
```

```
master_ip_online_change_script=/usr/local/bin/master_ip_online_change
```

```
#一旦 MHA 到 47 的监控之间出现问题, MHA Manager 将会尝试从 48 和 49 登录到 47。
```

```
secondary_check_script=/usr/local/bin/masterha_secondary_check -s 192.168.4.48 -s
192.168.4.49 --user=root --port=22 --master_host=192.168.4.47 --master_port=6133
```

```
#监控主节点时间间隔
```

```
ping_interval=3
```

```
#设置故障发生后关闭故障主机脚本（该脚本的主要作用是关闭主机放在发生脑裂，这里没有使用）
#shutdown_script=""
#数据库监控用户
user=root
password=ZXJYadmin123$
#复制用户
repl_user=root
repl_password=ZXJYadmin123$
#ssh 登录用户
ssh_user=root

[server1]
hostname=192.168.4.47
port=6133

[server2]
#设置为候选 master，如果设置该参数以后，发生主从切换以后将会将此从库提升为主库，即使这个主库不是集群中
事件最新的 slave
candidate_master=1
#默认情况下如果一个 slave 落后 master 100M 的 relay logs 的话，MHA 将不会选择该 slave 作为一个新的
master，因为对于这个 slave 的恢复需要花费很长时间，通过设置 check_repl_delay=0,MHA 触发切换在选择一
一个新的 master 的时候将会忽略复制延时，这个参数对于设置了 candidate_master=1 的主机非常有用，因为这个
候选主在切换的过程中一定是新的 master
check_repl_delay=0
hostname=192.168.4.48
port=6133

[server3]
hostname=192.168.4.49
port=6133
```



app50_47.cnf



app50_48.cnf

1.9 检查状态

检查 ssh 状态，成功！

masterha_check_ssh --conf=/etc/mha/app50.cnf


```
[root@mysql-03 ~]# masterha_check_ssh --conf=/etc/mha/app50.cnf
Mon Jun 1 12:07:57 2020 - [warning] Global configuration file /etc/masterha_default.cnf not found. Skipping.
Mon Jun 1 12:07:57 2020 - [info] Reading application default configuration from /etc/mha/app50.cnf..
Mon Jun 1 12:07:57 2020 - [info] Reading server configuration from /etc/mha/app50.cnf..
Mon Jun 1 12:07:57 2020 - [info] Starting SSH connection tests..
Mon Jun 1 12:07:58 2020 - [debug]
Mon Jun 1 12:07:57 2020 - [debug] Connecting via SSH from root@10.168.4.47(10.168.4.47:22) to root@10.168.4.48(10.168.4.48:22)..
Warning: Permanently added '10.168.4.47' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.168.4.48' (ECDSA) to the list of known hosts.
Mon Jun 1 12:07:58 2020 - [debug] ok.
Mon Jun 1 12:07:58 2020 - [debug] Connecting via SSH from root@10.168.4.47(10.168.4.47:22) to root@10.168.4.49(10.168.4.49:22)..
Warning: Permanently added '10.168.4.47' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.168.4.49' (ECDSA) to the list of known hosts.
Mon Jun 1 12:07:58 2020 - [debug] ok.
Mon Jun 1 12:07:59 2020 - [debug]
Mon Jun 1 12:07:58 2020 - [debug] Connecting via SSH from root@10.168.4.49(10.168.4.49:22) to root@10.168.4.47(10.168.4.47:22)..
Warning: Permanently added '10.168.4.49' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.168.4.47' (ECDSA) to the list of known hosts.
Mon Jun 1 12:07:59 2020 - [debug] ok.
Mon Jun 1 12:07:59 2020 - [debug] Connecting via SSH from root@10.168.4.49(10.168.4.49:22) to root@10.168.4.48(10.168.4.48:22)..
Warning: Permanently added '10.168.4.49' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.168.4.48' (ECDSA) to the list of known hosts.
Mon Jun 1 12:07:59 2020 - [debug] ok.
Mon Jun 1 12:07:59 2020 - [debug]
Mon Jun 1 12:07:58 2020 - [debug] Connecting via SSH from root@10.168.4.48(10.168.4.48:22) to root@10.168.4.47(10.168.4.47:22)..
Warning: Permanently added '10.168.4.48' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.168.4.47' (ECDSA) to the list of known hosts.
Mon Jun 1 12:07:58 2020 - [debug] ok.
Mon Jun 1 12:07:58 2020 - [debug] Connecting via SSH from root@10.168.4.48(10.168.4.48:22) to root@10.168.4.49(10.168.4.49:22)..
Warning: Permanently added '10.168.4.48' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.168.4.49' (ECDSA) to the list of known hosts.
Mon Jun 1 12:07:58 2020 - [debug] ok.
Mon Jun 1 12:07:59 2020 - [info] All SSH connection tests passed successfully.
```

检查集群的健康状态

失败原因大该有以下几点，这个总结下：

1 写 binlog 和复制过滤规则主从必须相同。

```
Mon Jan 14 20:20:23 2019 - [error][/usr/local/share/perl5/MHA/ServerManager.pm, ln467] binlog_ignore_db on master(information
_schema,mysql,performance_schema,test) must be the same as replicate_ignore_db on slaves(test).
Mon Jan 14 20:20:23 2019 - [warning] Bad Binlog/Replication filtering rules:
10.238.162.31 (current_master)
  Binlog_Do_DB:
  Binlog_Ignore_DB:
  Replicate_Do_DB:
  Replicate_Ignore_DB:
  Replicate_Do_Table:
  Replicate_Ignore_Table:
  Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:

10.238.162.33
  Binlog_Do_DB:
  Binlog_Ignore_DB: information_schema,mysql,performance_schema,test
  Replicate_Do_DB:
  Replicate_Ignore_DB: test
  Replicate_Do_Table:
  Replicate_Ignore_Table:
  Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:
```

2 找不到命令问题，解决方法，为命令建立软连接。

三个节点都要执行

```
ln -s /u01/mysql/bin/mysqlbinlog /usr/local/bin/
ln -s /u01/mysql/bin/mysqlbinlog /usr/bin/
ln -s /u01/mysql/bin/mysql /usr/local/bin/
```

```
ln -s /u01/mysql/bin/mysql /usr/bin/
```

```
Can't exec "mysqlbinlog": 没有那个文件或目录 at /usr/local/share/perl5/MHA/BinlogManager.pm line 106.
mysqlbinlog version command failed with rc 1:0, please verify PATH, LD_LIBRARY_PATH, and client options
at /usr/local/bin/apply_diff_relay_logs line 532.
Mon Jan 14 20:49:19 2019 - [error][/usr/local/share/perl5/MHA/MasterMonitor.pm, ln208] Slaves settings check failed!
Mon Jan 14 20:49:19 2019 - [error][/usr/local/share/perl5/MHA/MasterMonitor.pm, ln416] Slave configuration failed.
Mon Jan 14 20:49:19 2019 - [error][/usr/local/share/perl5/MHA/MasterMonitor.pm, ln427] Error happened on checking configurati
ons. at /usr/local/bin/masterha_check_repl line 48.
Mon Jan 14 20:49:19 2019 - [error][/usr/local/share/perl5/MHA/MasterMonitor.pm, ln525] Error happened on monitoring servers.
Mon Jan 14 20:49:19 2019 - [info] Got exit code 1 (Not master dead).
```

https://blog.csdn.net/qq_35209838

masterha_check_repl --conf=/etc/mha/app50.cnf

```
[root@mysql-03 ~]# masterha_check_repl --conf=/etc/mha/app50.cnf
Mon Jun 1 12:09:23 2020 - [warning] Global configuration file /etc/masterha_default.cnf
not found. Skipping.
Mon Jun 1 12:09:23 2020 - [info] Reading application default configuration from
/etc/mha/app50.cnf..
Mon Jun 1 12:09:23 2020 - [info] Reading server configuration from /etc/mha/app50.cnf..
Mon Jun 1 12:09:23 2020 - [info] MHA::MasterMonitor version 0.58.
Mon Jun 1 12:09:24 2020 - [info] GTID failover mode = 0
Mon Jun 1 12:09:24 2020 - [info] Dead Servers:
Mon Jun 1 12:09:24 2020 - [info] Alive Servers:
Mon Jun 1 12:09:24 2020 - [info] 192.168.4.47(192.168.4.47:6133)
Mon Jun 1 12:09:24 2020 - [info] 192.168.4.48(192.168.4.48:6133)
Mon Jun 1 12:09:24 2020 - [info] 192.168.4.49(192.168.4.49:6133)
Mon Jun 1 12:09:24 2020 - [info] Alive Slaves:
Mon Jun 1 12:09:24 2020 - [info] 192.168.4.48(192.168.4.48:6133) Version=5.7.21-log
(oldest major version between slaves) log-bin:enabled
Mon Jun 1 12:09:24 2020 - [info] Replicating from 192.168.4.47(192.168.4.47:6133)
Mon Jun 1 12:09:24 2020 - [info] Primary candidate for the new Master (candidate_master
is set)
Mon Jun 1 12:09:24 2020 - [info] 192.168.4.49(192.168.4.49:6133) Version=5.7.21-log
(oldest major version between slaves) log-bin:enabled
Mon Jun 1 12:09:24 2020 - [info] Replicating from 192.168.4.47(192.168.4.47:6133)
Mon Jun 1 12:09:24 2020 - [info] Current Alive Master: 192.168.4.47(192.168.4.47:6133)
Mon Jun 1 12:09:24 2020 - [info] Checking slave configurations..
Mon Jun 1 12:09:24 2020 - [info] read_only=1 is not set on slave
192.168.4.48(192.168.4.48:6133).
Mon Jun 1 12:09:24 2020 - [info] read_only=1 is not set on slave
192.168.4.49(192.168.4.49:6133).
Mon Jun 1 12:09:24 2020 - [info] Checking replication filtering settings..
Mon Jun 1 12:09:24 2020 - [info] binlog_do_db= , binlog_ignore_db=
Mon Jun 1 12:09:24 2020 - [info] Replication filtering check ok.
Mon Jun 1 12:09:24 2020 - [info] GTID (with auto-pos) is not supported
Mon Jun 1 12:09:24 2020 - [info] Starting SSH connection tests..
Mon Jun 1 12:09:26 2020 - [info] All SSH connection tests passed successfully.
Mon Jun 1 12:09:26 2020 - [info] Checking MHA Node version..
```

```
Mon Jun 1 12:09:26 2020 - [info] Version check ok.
Mon Jun 1 12:09:26 2020 - [info] Checking SSH publickey authentication settings on the
current master..
Warning: Permanently added '192.168.4.47' (ECDSA) to the list of known hosts.
Mon Jun 1 12:09:26 2020 - [info] HealthCheck: SSH to 192.168.4.47 is reachable.
Mon Jun 1 12:09:26 2020 - [info] Master MHA Node version is 0.58.
Mon Jun 1 12:09:26 2020 - [info] Checking recovery script configurations on
192.168.4.47(192.168.4.47:6133)..
Mon Jun 1 12:09:26 2020 - [info] Executing command: save_binary_logs --command=test
--start_pos=4 --binlog_dir=/u01/data --output_file=/u01/mha_tmp/save_binary_logs_tes
t --manager_version=0.58 --start_file=mysql-bin.000020 Mon Jun 1 12:09:26 2020 - [info]
Connecting to root@192.168.4.47(192.168.4.47:22)..
Warning: Permanently added '192.168.4.47' (ECDSA) to the list of known hosts.
    Creating /u01/mha_tmp if not exists..    ok.
    Checking output directory is accessible or not..
    ok.
    Binlog found at /u01/data, up to mysql-bin.000020
Mon Jun 1 12:09:27 2020 - [info] Binlog setting check done.
Mon Jun 1 12:09:27 2020 - [info] Checking SSH publickey authentication and checking recovery
script configurations on all alive slave servers..
Mon Jun 1 12:09:27 2020 - [info] Executing command: apply_diff_relay_logs --command=test
--slave_user='root' --slave_host=192.168.4.48 --slave_ip=192.168.4.48 --slave_po
rt=6133 --workdir=/u01/mha_tmp --target_version=5.7.21-log --manager_version=0.58
--relay_dir=/u01/data --current_relay_log=mysql-02-relay-bin.000062
--slave_pass=xxxMon Jun 1 12:09:27 2020 - [info] Connecting to
root@192.168.4.48(192.168.4.48:22)..
Warning: Permanently added '192.168.4.48' (ECDSA) to the list of known hosts.
    Checking slave recovery environment settings..
        Relay log found at /u01/data, up to mysql-02-relay-bin.000062
        Temporary relay log file is /u01/data/mysql-02-relay-bin.000062
        Checking if super_read_only is defined and turned on.. not present or turned off,
ignoring.
        Testing mysql connection and privileges..
mysql: [warning] Using a password on the command line interface can be insecure.
done.
    Testing mysqlbinlog output.. done.
    Cleaning up test file(s).. done.
Mon Jun 1 12:09:27 2020 - [info] Executing command: apply_diff_relay_logs --command=test
--slave_user='root' --slave_host=192.168.4.49 --slave_ip=192.168.4.49 --slave_po
rt=6133 --workdir=/u01/mha_tmp --target_version=5.7.21-log --manager_version=0.58
--relay_dir=/u01/data --current_relay_log=mysql-03-relay-bin.000062
--slave_pass=xxxMon Jun 1 12:09:27 2020 - [info] Connecting to
root@192.168.4.49(192.168.4.49:22)..
Warning: Permanently added '192.168.4.49' (ECDSA) to the list of known hosts.
```

```
Checking slave recovery environment settings..
  Relay log found at /u01/data, up to mysql-03-relay-bin.000062
  Temporary relay log file is /u01/data/mysql-03-relay-bin.000062
  Checking if super_read_only is defined and turned on.. not present or turned off,
  ignoring.
  Testing mysql connection and privileges..
mysql: [warning] Using a password on the command line interface can be insecure.
done.
  Testing mysqlbinlog output.. done.
  Cleaning up test file(s).. done.
Mon Jun 1 12:09:27 2020 - [info] Slaves settings check done.
Mon Jun 1 12:09:27 2020 - [info]
192.168.4.47(192.168.4.47:6133) (current master)
+--192.168.4.48(192.168.4.48:6133)
+--192.168.4.49(192.168.4.49:6133)

Mon Jun 1 12:09:27 2020 - [info] Checking replication health on 192.168.4.48..
Mon Jun 1 12:09:27 2020 - [info] ok.
Mon Jun 1 12:09:27 2020 - [info] Checking replication health on 192.168.4.49..
Mon Jun 1 12:09:27 2020 - [info] ok.
Mon Jun 1 12:09:27 2020 - [warning] master_ip_failover_script is not defined.
Mon Jun 1 12:09:27 2020 - [warning] shutdown_script is not defined.
Mon Jun 1 12:09:27 2020 - [info] Got exit code 0 (Not master dead).

MySQL Replication Health is OK.
```

1.10 启停 MHA manager 监控

```
#查看 MHA manager 监控状态, 这里没有运行
# masterha_check_status --conf=/etc/mha/app50.cnf
App50 is stopped(2:NOT_RUNNING).

#启动 MHA 监控 --remove_dead_master_conf --ignore_last_failover
nohup masterha_manager --conf=/etc/mha/app50.cnf --remove_dead_master_conf
--ignore_last_failover &

# 停止 MHA 监控
masterha_stop --conf=/etc/mha/app50.cnf
```

1.11 主从恢复脚本

一节点脚本



recover.sh

二节点脚本



recover.sh

1.12 创建自动清除 relay log 脚本

MHA 在发生切换的过程中，从库的恢复过程中依赖于 relay log 的相关信息，所以这里要将 relay log 的自动清除设置为 OFF，采用手动清除 relay log 的方式。在默认情况下，从服务器上的中继日志会在 SQL 线程执行完毕后被自动删除。但是在 MHA 环境中，这些中继日志在恢复其他从服务器时可能会被用到，因此需要禁用中继日志的自动删除功能。定期清除中继日志需要考虑到复制延时的问题。在 ext3 的文件系统下，删除大的文件需要一定的时间，会导致严重的复制延时。为了避免复制延时，需要暂时为中继日志创建硬链接，因为在 linux 系统中通过硬链接删除大文件速度会很快。（在 mysql 数据库中，删除大表时，通常也采用建立硬链接的方式）

MHA 节点中包含了 pure_relay_logs 命令工具，它可以为中继日志创建硬链接，执行 SET GLOBAL relay_log_purge=1, 等待几秒钟以便 SQL 线程切换到新的中继日志，再执行 SET GLOBAL relay_log_purge=0。

清理日志脚本

```
[root@mysql-03 ~]# cat /u01/mha/purge_relay_log.sh
#!/bin/bash
#数据库用户名密码端口
user=root
passwd=ZXJYadmin123$
port=6133
#脚本日志存放路径
log_dir='/u01/mha/app50'
#指定创建 relay log 的硬链接的位置，默认是/var/tmp。由于系统不同分区创建硬链接文件会失败，故需要执行硬链接具体位置，成功执行脚本后，硬链接的中继日志文件被删除。
work_dir='/u01/mha'
```

```
#删除中继日志脚本
purge='/usr/local/bin/purge_relay_logs'

if [ ! -d $log_dir ]
then
    mkdir $log_dir -p
fi

#--disable_relay_log_purge : 默认情况下, 如果 relay_log_purge=1, 脚本会什么都不清理, 自动退出。
通过设定这个参数, 当 relay_log_purge=1 的情况下会将 relay_log_purge 设置为 0。清理
relay log 之后, 最后将参数设置为 OFF。$purge --user=$user --password=$passwd
--disable_relay_log_purge --port=$port --workdir=$work_dir >>
$log_dir/purge_relay_logs.log 2>&1
```

并授予可执行权限

```
chmod +x purge_relay_log.sh
```

添加至定时任务

```
00 03 * * * /bin/bash /u01/mha/purge_relay_log.sh
```

手动执行脚本查看输出日志:

```
[root@mysql-03 ~]# /bin/bash /u01/mha/purge_relay_log.sh
[root@mysql-03 app50]# cat /u01/mha/app1/purge_relay_logs.log
2020-04-24 11:19:02: purge_relay_logs script started.
Found relay_log.info: /u01/data/relay-log.info
Opening /u01/data/cdht8-relay-bin.000001 ..
Opening /u01/data/cdht8-relay-bin.000002 ..
Opening /u01/data/cdht8-relay-bin.000003 ..
Opening /u01/data/cdht8-relay-bin.000004 ..
Executing SET GLOBAL relay_log_purge=1; FLUSH LOGS; sleeping a few seconds so that SQL
thread can delete older relay
log files (if it keeps up); SET GLOBAL relay_log_purge=0; .. ok.2020-04-24 11:19:05: All
relay log purging operations succeeded.
```

1.13使用 keepalived 引入 VIP

通过 keepalived 的方式管理 VIP。

安装 keepalived yum install keepalived -y

北京首钢自动化信息技术有限公司

邮编: 100041 TEL: 0315-7706587

Mysql-01 的配置

```
[root@mysql-01 ~]# cat /etc/keepalived/keepalived.conf
```

```
! Configuration File for keepalived
```

```
global_defs {
    router_id mysql-50
}

vrrp_instance VI_1 {
    state BACKUP
    interface eth0
    virtual_router_id 50
    priority 100
    advert_int 1
    nopreempt
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    unicast_src_ip 192.168.4.47
    unicast_peer {
        192.168.4.48
    }
    virtual_ipaddress {
        192.168.4.50/24
    }
}

virtual_server 192.168.4.50 6133 {
    delay_loop 2
    lb_algo rr
    lb_kind DR
    persistence_timeout 60

    real_server 192.168.4.47 6133 {
        weight 3
        notify_down /etc/keepalived/mysql.sh
        TCP_CHECK {
            connect_timeout 3
        }
    }
}
```

```
[root@mysql-01 ~]# cat /etc/keepalived/mysql.sh
#!/bin/bash
pkill keepalived
ifconfig eth0 down && ifconfig eth0 up
```

Mysql-02 的配置

```
[root@mysql-02 ~]# cat /etc/keepalived/keepalived.conf
! Configuration File for keepalived

global_defs {
    router_id mysql-50
}

vrrp_instance VI_1 {
    state BACKUP
    interface eth0
    virtual_router_id 50
    priority 99
    advert_int 1
    nopreempt
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    unicast_src_ip 192.168.4.48
    unicast_peer {
        192.168.4.47
    }
    virtual_ipaddress {
        192.168.4.50/24
    }
}

virtual_server 192.168.4.50 6133 {
    delay_loop 2
    lb_algo rr
    lb_kind DR
    persistence_timeout 60

    real_server 192.168.4.48 6133 {
        weight 3
        notify_down /etc/keepalived/mysql.sh
        TCP_CHECK {
```



```
        connect_timeout 3
    }
}
}
[root@mysql-02 ~]# cat /etc/keepalived/mysql.sh
#!/bin/bash
pkill keepalived
ifconfig eth0 down && ifconfig eth0 up
```

注意：要添加/etc/keepalived/mysql.sh 可执行权限

```
[root@mysql-02 ~]# chmod +x /etc/keepalived/mysql.sh
```